

Robô Seguidor de Linha

Desenvolvimento do código

Este documento resume como o firmware do robô foi desenvolvido: a escolha da plataforma, a estratégia de controle e os problemas encontrados ao longo dos testes, junto com a solução adotada para cada um.

1. Escolha da plataforma

O uso de um microcontrolador STM32 foi uma escolha natural para a equipe, já que todos os integrantes tinham familiaridade com a família e com o ambiente STM32CubeIDE. Entre as placas disponíveis, a escolhida foi a **Black Pill (STM32F401CCU6)**, no lugar da Blue Pill (STM32F103), por três motivos:

- **DMA:** o conversor analógico-digital trabalha em modo contínuo e o DMA copia as leituras dos 8 sensores para a memória sozinho, sem ocupar o processador. O laço de controle sempre encontra a leitura mais recente pronta.
- **Entradas analógicas:** a placa tem entradas de ADC suficientes para ligar os 8 sensores da barra, um por canal, em uma única varredura.
- **Clock maior:** 84 MHz contra 72 MHz da Blue Pill. O STM32F401 também tem unidade de ponto flutuante (FPU), o que deixa o cálculo do PID em `float` rápido.

2. Estratégia de controle: PID

O código foi desenvolvido diretamente com um controlador **PID**. O robô usa uma barra de sensores da **Pololu** com saídas analógicas, que permite calcular a posição da linha de forma contínua, de 0 a 7000 (centro em 3500), por média ponderada dos sensores. Com uma medida de erro contínua e proporcional ao desvio, o PID é a técnica natural; métodos mais simples, como ligar e desligar os motores conforme o sensor que vê a linha, desperdiçariam essa informação.

```
posição = Σ (leitura[i] · peso[i]) / Σ leitura[i] // pesos 0, 1000, ..., 7000
erro     = 3500 - posição
saída    = Kp · erro + Ki · Σerro + Kd · (erro - erro_anterior)
```

O cálculo roda a cada 2 ms (500 Hz), comandado por um timer do microcontrolador.

3. Primeiro problema: motores não lineares

Logo nos primeiros testes os motores não respondiam de forma linear ao comando de velocidade. Havia uma **zona morta considerável**: abaixo de cerca de 30% de duty do PWM as rodas simplesmente não giravam, e acima disso a velocidade subia de repente. Com isso, pequenas correções do PID não tinham efeito nenhum e correções um pouco maiores eram exageradas.

A solução foi uma **compensação de zona morta** na camada dos motores. O comando, numa escala de 0 a 1000, é remapeado para começar exatamente no ponto em que a roda passa a girar:

```
|comando| < 20      → 0 (freio)
|comando| em 1..1000 → 300 + (|comando| - 1) · 700 / 999
```

Para descobrir esse ponto (300, ou 30%), o firmware ganhou um modo de medição que sobe o PWM dos motores aos poucos e informa cada passo, até a roda começar a girar. Com a zona morta compensada, o sistema passou a responder de forma aproximadamente linear e o PID passou a ter efeito previsível.

4. Refino do PID e o problema dos valores negativos

Com a linearidade resolvida, a equipe passou a refinar os ganhos do PID. O PID já era suficiente para seguir a linha, mas havia um problema: em curvas, a correção era grande o bastante para levar o comando de uma das rodas a **valores negativos**. A roda passava a girar para trás e o robô chegava a andar de ré, o que dificultava muito seguir a linha.

A solução foi **não permitir comando negativo nos motores**. Na curva, a roda de fora mantém a velocidade base e a roda de dentro é desacelerada até, no máximo, parar:

```
roda de fora = velocidade base  
roda de dentro = max(velocidade base - 2·|correção|, 0)
```

Com essa mudança o robô passou a seguir a linha de forma bem mais fluida.

5. Mudanças bruscas e velocidade nas curvas

Ainda restavam dois problemas. O primeiro: as **mudanças bruscas de comando** entre um ciclo e outro faziam as rodas trocarem de velocidade de forma abrupta, o que fazia o robô tremer e perder muito tempo. O segundo: pela forma como o cálculo do PID era somado às rodas, o robô **ganhava velocidade nas curvas**, porque a roda de fora acelerava acima da velocidade base justamente no trecho mais difícil.

As duas soluções adotadas:

- **Limite de velocidade nas curvas:** a roda de fora nunca passa da velocidade base; toda a correção é feita reduzindo a roda de dentro. Assim a curva nunca é mais rápida que a reta. O firmware também tem uma opção para reduzir a velocidade base proporcionalmente ao erro.
- **Função de rampa:** o comando de cada roda só pode mudar um tanto por ciclo de 2 ms: até 40 unidades ao acelerar (0 a 100% em 50 ms) e até 60 ao desacelerar (100% a 0 em cerca de 34 ms). A desaceleração é mais rápida para o robô não entrar atrasado nas curvas.

Com o limite nas curvas e a rampa, o robô alcançou uma **movimentação tranquila**, sem os trancos e sem a perda de tempo dos testes anteriores.

6. Telemetria com o ESP32-C3

Para acompanhar o robô sem precisar regravar o firmware a cada teste, a equipe adicionou um **ESP32-C3** usado **apenas para telemetria e ajuste**; ele não participa do controle. O ESP32-C3 cria uma rede Wi-Fi com uma página web e conversa com o STM32 pela serial (UART). Pela página é possível:

- acompanhar ao vivo a leitura de cada sensor e a posição calculada da linha;
- calibrar os sensores arrastando o robô sobre a linha;
- mudar Kp, Ki, Kd e a velocidade base em tempo real, o que tornou o refino do PID muito mais rápido;
- testar cada motor separadamente e parar o robô a qualquer momento.

7. Resumo da evolução

Etapa	Problema	Solução
1	Motores não lineares, com zona morta de ~30%	Compensação de zona morta e modo de medição
2	Valores negativos do PID faziam o robô andar de ré	Comando dos motores limitado a zero (sem ré)
3	Mudanças bruscas de movimento e perda de tempo	Função de rampa na aceleração e na desaceleração
4	Robô ganhava velocidade nas curvas	Roda de fora limitada à velocidade base
–	Ajuste lento do PID e dos sensores	Telemetria e ajuste em tempo real pelo ESP32-C3